Software Design Specification

HYBRID_LIB_ADA

Document Control

Field	Value
Author(s)	Michael Gardner
Version	2.0.1
Status	Released
Status Date	2025-12-11
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2025 Michael Gardner, A Bit of Help, Inc.

Project Profile

Field	Value
Variant	library
Library Role	enterprise
Assurance	spark-targeted
Execution	sequential
Parallelism	none
Platform	desktop, server, embedded
Execution Environment	linux, macos, windows, ada-runtime
Processor Architecture	amd64, arm64, arm32
Deployment	native

Table of Contents

1. Introduction ... 1
   1.1. Purpose ... 1
   1.2. Scope ... 1
   1.3. References ... 1
2. Architectural Overview ... 1
   2.1. Layer Architecture ... 1
   2.2. Dependency Rules ... 2
   2.3. Hexagonal Pattern ... 2
3. Package Structure ... 2
   3.1. Directory Layout ... 2
   3.2. Package Descriptions ... 3
      3.2.1. Domain Layer ... 3
      3.2.2. Application Layer ... 4
      3.2.3. Infrastructure Layer ... 4
      3.2.4. API Layer ... 4
4. Type Definitions ... 4
   4.1. Domain Types ... 4
      4.1.1. Error_Kind ... 4
      4.1.2. Error_Type ... 4
      4.1.3. Result (Generic) ... 4
      4.1.4. Person ... 5
   4.2. Application Types ... 5
      4.2.1. Greet_Command ... 5
      4.2.2. Writer Port ... 5
   4.3. API Types ... 5
5. Design Patterns ... 5
   5.1. Static Dependency Injection ... 5
   5.2. Three-Package API Pattern ... 6
   5.3. Result Monad Pattern ... 6
6. Error Handling Strategy ... 6
   6.1. Error Propagation ... 6
   6.2. No Exceptions Policy ... 6
   6.3. Exception Boundary Specification ... 7
      6.3.1. Required Pattern: Functional.Try at Boundaries ... 7
      6.3.2. Prohibited Pattern: Manual Exception Handlers ... 8
      6.3.3. Validation Enforcement ... 8
7. Build Configuration ... 8
   7.1. GPR Projects ... 8
   7.2. Build Profiles ... 8
8. Design Decisions ... 8
   8.1. API.Operations as Child vs Sibling ... 8
   8.2. No Heap Allocation ... 8
   8.3. Static vs Dynamic Polymorphism ... 8
9. Appendices ... 9
   9.1. Package Dependency Notes ... 9
   9.2. Change History ... 9

1. Introduction

1.1. Purpose

This Software Design Specification (SDS) describes the internal architecture, package structure, key type definitions, build strategy, and design decisions for **HYBRID_LIB_ADA**.

1.2. Scope

This document covers:

- the 4-layer hybrid library architecture,
- the three-package API pattern,
- package hierarchy and dependency rules,
- core type definitions and contracts,
- static dependency injection via Ada generics,
- SPARK verification boundaries, and
- the layered exception-boundary enforcement strategy.

1.3. References

- Software Requirements Specification (SRS).
- `docs/guides/all_about_our_api.md`.
- `docs/guides/architecture_enforcement.md`.
- Supporting UML diagrams maintained in `docs/diagrams/`.
- Ada 2022 Reference Manual.
- SPARK Reference Manual.

2. Architectural Overview

2.1. Layer Architecture

HYBRID_LIB_ADA uses the shared hybrid architecture family and the same inner core as the application variant: **Domain → Application → Infrastructure**. The library profile differs only at the outer boundary, where it uses the API pattern instead of Presentation + Bootstrap.

Layer / Boundary	Purpose	Depends On
API Facade (`Hybrid_Lib_Ada.API`)	Thin public facade; re-exports public types and delegates operations to the default composition root.	Application + Domain
API Composition Root (`API.Desktop`)	Default desktop/server composition root; wires infrastructure adapters into the generic operations surface.	All layers
API Operations (`API.Operations`)	SPARK-safe operations surface parameterized by port implementations.	Application + Domain
Infrastructure	Adapters for I/O and external integration; implements application-owned ports.	Application + Domain
Application	Use cases, commands, and port definitions.	Domain

Layer / Boundary	Purpose	Depends On
Domain	Pure business concepts, value objects, and error types.	Nothing

Detailed architecture guidance is maintained in `docs/guides/`. Supporting UML diagrams are maintained in `docs/diagrams/`.

2.2. Dependency Rules

The API layer has **two distinct areas** with different dependency rules.

Component	May Depend On
Domain	Nothing (zero dependencies).
Application	Domain only.
Infrastructure	Application and Domain.
API facade (`api/`)	Application + Domain only.
API composition roots (`api/desktop/`)	All layers, including Infrastructure.

The API facade (`Hybrid_Lib_Ada.API`, `API.Operations`) provides the public interface and does not import Infrastructure. The composition root (`API.Desktop`) is the intentional exception that wires concrete adapters to the generic operations surface.

Architecture conformance is enforced through layered controls:
- project/build configuration constraints where applicable,
- active validation using `scripts/arch_guard.py`,
- execution of `scripts/arch_guard.py` during normal developer build workflow, and
- execution of `scripts/arch_guard.py` in CI/CD before non-conforming changes are accepted.

2.3. Hexagonal Pattern

The Application layer defines ports and use cases. Infrastructure adapters implement outbound ports. Domain and Application remain independent of concrete I/O concerns.

Element	Role
Application.Usecase.Greet	Orchestrates the greeting use case using domain validation and an outbound Writer port.
Application.Port.Outbound.Writer	Application-owned outbound port contract.
Infrastructure.Adapter.Console_Writer	Concrete infrastructure adapter implementing the Writer port.

3. Package Structure

3.1. Directory Layout

```
src/
├── hybrid_lib_ada.ads              # Root package
├── version/
│   └── hybrid_lib_ada-version.ads  # Version information
│
├── domain/
│   ├── domain.ads                  # Domain layer root
│   ├── domain-unit.ads             # Unit type
```

```
│   ├── error/
│   │   ├── domain-error.ads        # Error type definition
│   │   └── domain-error-result.ads # Generic Result monad
│   └── value_object/
│       ├── domain-value_object.ads
│       ├── domain-value_object-option.ads
│       └── domain-value_object-person.ads
│
├── application/
│   ├── application.ads
│   ├── error/
│   │   └── application-error.ads   # Re-exports Domain.Error
│   ├── command/
│   │   ├── application-command.ads
│   │   └── application-command-greet.ads
│   ├── port/
│   │   ├── application-port.ads
│   │   ├── inbound/
│   │   │   └── application-port-inbound.ads
│   │   └── outbound/
│   │       ├── application-port-outbound.ads
│   │       └── application-port-outbound-writer.ads
│   └── usecase/
│       ├── application-usecase.ads
│       └── application-usecase-greet.ads
│
├── infrastructure/
│   ├── infrastructure.ads
│   └── adapter/
│       ├── infrastructure-adapter.ads
│       └── infrastructure-adapter-console_writer.ads
│
└── api/
    ├── hybrid_lib_ada-api.ads
    ├── hybrid_lib_ada-api.adb
    ├── operations/
    │   ├── hybrid_lib_ada-api-operations.ads
    │   └── hybrid_lib_ada-api-operations.adb
    └── desktop/
        ├── hybrid_lib_ada-api-desktop.ads
        └── hybrid_lib_ada-api-desktop.adb
```

3.2. Package Descriptions

3.2.1. Domain Layer

Package	Purpose	SPARK
`Domain`	Layer root.	On
`Domain.Unit`	Unit type for void-like operations.	On
`Domain.Error`	Error type with kind and bounded message.	On
`Domain.Error.Result`	Generic Result[T] monad.	On
`Domain.Value_Object`	Value object root.	On
`Domain.Value_Object.Option`	Option[T] monad.	On
`Domain.Value_Object.Person`	Person value object.	On

3.2.2. Application Layer

Package	Purpose	SPARK
`Application`	Layer root.	On
`Application.Error`	Re-exports Domain.Error.	On
`Application.Command`	Command root.	On
`Application.Command.Greet`	Greeting command DTO.	On
`Application.Port`	Port root.	On
`Application.Port.Outbound.Writer`	Writer port definition.	On
`Application.Usecase.Greet`	Greeting use case.	On

3.2.3. Infrastructure Layer

Package	Purpose	SPARK
`Infrastructure`	Layer root.	Off
`Infrastructure.Adapter`	Adapter root.	Off
`Infrastructure.Adapter.Console_Writer`	Console output adapter.	Off

3.2.4. API Layer

Package	Purpose	SPARK
`Hybrid_Lib_Ada`	Library root.	Off
`Hybrid_Lib_Ada.Version`	Version information.	Off
`Hybrid_Lib_Ada.API`	Public facade.	Off
`Hybrid_Lib_Ada.API.Operations`	SPARK-safe operations surface.	On
`Hybrid_Lib_Ada.API.Desktop`	Desktop composition root.	Off

4. Type Definitions

4.1. Domain Types

4.1.1. Error_Kind

```
type Error_Kind is
  (Validation_Error,
   Parse_Error,
   Not_Found_Error,
   IO_Error,
   Internal_Error);
```

4.1.2. Error_Type

```
type Error_Type is record
   Kind    : Error_Kind;
   Message : Error_Strings.Bounded_String;  -- Max 512 chars
end record;
```

4.1.3. Result (Generic)

```
generic
   type T is private;
package Domain.Error.Result.Generic_Result is
   type Result is private;
```

```
   function Ok (Value : T) return Result;
   function Error (Kind : Error_Kind; Message : String) return Result;

   function Is_Ok (R : Result) return Boolean;
   function Is_Error (R : Result) return Boolean;
   function Value (R : Result) return T with Pre => Is_Ok (R);
   function Error_Info (R : Result) return Error_Type with Pre => Is_Error (R);
end Generic_Result;
```

4.1.4. Person

```
type Person is private;

function Create (Name : String) return Person_Result.Result;
function Get_Name (P : Person) return String;
function Is_Valid_Person (P : Person) return Boolean;
```

4.2. Application Types

4.2.1. Greet_Command

```
type Greet_Command is private;

function Create (Name : String) return Greet_Command;
function Get_Name (Cmd : Greet_Command) return String;
```

4.2.2. Writer Port

```
generic
   with function Write (Message : String) return Unit_Result.Result;
package Generic_Writer is
   function Write_Message (Message : String) return Unit_Result.Result
   renames Write;
end Generic_Writer;
```

4.3. API Types

All public types are re-exported from `Hybrid_Lib_Ada.API`.

```
subtype Person_Type is Domain.Value_Object.Person.Person;
subtype Greet_Command is Application.Command.Greet.Greet_Command;
subtype Error_Type is Domain.Error.Error_Type;
subtype Error_Kind is Domain.Error.Error_Kind;
```

5. Design Patterns

5.1. Static Dependency Injection

HYBRID_LIB_ADA uses Ada generics for static (compile-time) dependency injection.

```
-- 1. Port defines generic signature
package Generic_Writer is ...

-- 2. Use case or operations package is generic, parameterized by Writer
package Application.Usecase.Greet is ...

-- 3. Composition root instantiates with concrete adapter
package Console_Ops is new Hybrid_Lib_Ada.API.Operations
  (Writer => Infrastructure.Adapter.Console_Writer.Write);
```

Benefit	Description
Zero runtime overhead	Monomorphization occurs at compile time.
SPARK compatible	No runtime dispatching is required.
Type safe	The compiler verifies contracts and instantiations.
Testable	Mock writers and alternate composition roots are easy to provide.

5.2. Three-Package API Pattern

The library profile uses a three-package API pattern:

- `Hybrid_Lib_Ada.API` as the public facade,
- `Hybrid_Lib_Ada.API.Desktop` as the default composition root,
- `Hybrid_Lib_Ada.API.Operations` as the SPARK-safe operations surface.

Package	Design Role	SPARK
`Hybrid_Lib_Ada.API`	Thin public facade; re-exports public types and delegates to the default composition root.	Off
`Hybrid_Lib_Ada.API.Desktop`	Composition root for desktop/server environments; wires adapters into operations.	Off
`Hybrid_Lib_Ada.API.Operations`	Generic operations surface parameterized by outbound ports; safe verification boundary.	On

5.3. Result Monad Pattern

All fallible operations return `Result[T]` values rather than propagating expected failures as exceptions.

```
function Create (Name : String) return Person_Result.Result;
-- Returns Ok(Person) or Error(Validation_Error, "message")

function Greet (Cmd : Greet_Command) return Unit_Result.Result;
-- Returns Ok(Unit) or Error(IO_Error, "message")
```

6. Error Handling Strategy

6.1. Error Propagation

Errors flow through use-case orchestration as Result values.

```
function Execute (Cmd : Greet_Command) return Unit_Result.Result is
   Person_Res : constant Person_Result.Result := Person.Create (Get_Name (Cmd));
begin
   if Person_Result.Is_Error (Person_Res) then
      return Unit_Result.Error (...);
   end if;

   return Writer (Format_Greeting (Person_Result.Value (Person_Res)));
end Execute;
```

6.2. No Exceptions Policy

Situation	Handling
Validation failure	Return Error result.
I/O failure	Return Error result.

Situation	Handling
Unexpected error at infrastructure boundary	Convert to `Internal_Error` result at the boundary.
Programmer error in core layers	Fail loudly; treat as a bug to fix rather than a normal runtime condition.

6.3. Exception Boundary Specification

This project uses a layered exception-boundary policy.

Layer	Functional.Try	Manual `exception`	Rationale
Infrastructure	Required at external boundaries	Forbidden	Must convert external exceptions to Result values consistently.
Domain	N/A	Forbidden	Pure core logic; works with Result values only.
Application	N/A	Forbidden	Use-case orchestration; works with Result values only.
API	N/A	Forbidden	Public facade; returns Result values to consumers.
Test	Optional	Allowed	Tests may deliberately verify exception scenarios.

6.3.1. Required Pattern: Functional.Try at Boundaries

All Infrastructure adapters that perform I/O or call external APIs shall use `Functional.Try.Map_To_Result` or `Functional.Try.Map_To_Result_With_Param`.

```
with Functional.Try.Map_To_Result_With_Param;

package body Infrastructure.Adapter.Console_Writer is
   -- Inner action may raise exceptions
   function Write_Action (Message : String) return Unit_Result.Result is
   begin
      Ada.Text_IO.Put_Line (Message);
      return Unit_Result.Ok (Unit_Value);
   end Write_Action;

   -- Error factory converts exception mapping into domain error
   function Make_Error (Kind : Error_Kind; Message : String)
     return Unit_Result.Result is
   begin
      return Unit_Result.Error (Kind, Message);
   end Make_Error;

   package Try_Write is new Functional.Try.Map_To_Result_With_Param (...);
   Mappings : constant Try_Write.Mapping_Array := [...];

   function Write (Message : String) return Unit_Result.Result is
   begin
      return Try_Write.Run (Message, Mappings);
```

```
   end Write;
end Infrastructure.Adapter.Console_Writer;
```

6.3.2. Prohibited Pattern: Manual Exception Handlers

Manual `exception` handlers in infrastructure adapters are forbidden because they create inconsistent error mapping, weaken auditability, and separate exception conversion from the shared policy.

6.3.3. Validation Enforcement

Release preparation and developer workflow validate these rules:

1. Infrastructure code shall use `Functional.Try` where boundary I/O exists.
2. Infrastructure code shall not contain manual `exception` handlers.
3. Domain/Application/API code shall not use the `exception` keyword.
4. Test code is exempt.

7. Build Configuration

7.1. GPR Projects

Project	Purpose
`hybrid_lib_ada.gpr`	Public library with `Library_Interface` restrictions.
`hybrid_lib_ada_internal.gpr`	Internal unrestricted project used for tests and examples.

7.2. Build Profiles

Profile	Target	Features
standard	Desktop/server	Full feature set.
embedded	Ravenscar embedded	Tasking-safe embedded profile.
baremetal	Minimal runtime	Zero-footprint style profile.

8. Design Decisions

8.1. API.Operations as Child vs Sibling

Decision: use `API.Operations` as a child package instead of a sibling package.

Rationale: the child-package hierarchy is more idiomatic Ada, preserves a clean public API structure, and supports the design without losing SPARK value.

8.2. No Heap Allocation

Decision: use bounded strings and stack-oriented allocation strategies.

Rationale: this supports embedded compatibility, deterministic behavior, and SPARK-friendly design constraints.

8.3. Static vs Dynamic Polymorphism

Decision: use static polymorphism via generics.

Rationale: this supports SPARK compatibility, zero runtime overhead, and compile-time type safety.

9. Appendices

9.1. Package Dependency Notes

The package dependency structure reflects the same hybrid core as the application profile, with the API pattern providing the library-specific outer boundary. Supporting UML diagrams are maintained in `docs/diagrams/`.

9.2. Change History

Version	Date	Author	Changes
2.0.1	2025-12-11	Michael Gardner	Added Section 6.3 Exception Boundary Specification with architecture diagram, layer rules, and required/prohibited patterns.
2.0.0	2025-12-09	Michael Gardner	Complete regeneration for v2.0.0; corrected Error_Kind (5 values); updated package structure.
1.0.0	2025-11-29	Michael Gardner	Initial release.